The Pollis Guide

How to use Pollis, one topic at a time

How this guide works

Each topic takes two facing pages: what the feature is for, where to find it, the steps, its limits, and what to do when something goes wrong. Read the topics in any order.

Menu paths are written **Compose > Collaboration > Join Session...**, and buttons as they appear, such as **Save As...**. A setting is named by its id, such as `pollis.collaboration.signalling`, which you can paste into the search box of **Settings**.

This guide describes Pollis as it is on 16 October 2026. What is planned but not available yet is said to be not available.

Getting Pollis

Pollis is free and open source, for Windows, macOS and Linux: github.com/saragga/pollis/releases/latest.
Questions and problems: **Help > Report Issue**, or contact@pollis.org.

Contents

2 Getting Started
4 Inside a Panel
6 Data and Databases
8 Customisation
10 Toolboxes
13 Replication, Methods and Workflows
14 Replication
16 Methods
18 Pollis Packages
20 Reports
22 Practice and Learning Activities
24 Collaboration
26 Orchestration

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 16 October 2026

Getting Started

Welcome page · Julia REPL

Installing Pollis and Julia, the first launch, and how the Julia packages a panel needs are installed.

Pollis is a single application with no account and no licence key. It writes Julia code; Julia, which runs that code, is installed separately, from the Welcome page, in a few minutes.

Install Pollis

Download the file for your system from github.com/saragga/pollis/releases/latest.

- **macOS** (Apple silicon), `Pollis-darwin-arm64.zip`:
  1. Unzip it and move `Pollis.app` to `Applications`.
  2. In Terminal, run `xattr -cr /Applications/Pollis.app`.
  3. Open Pollis.
- **Windows** (x64), `PollisSetup-x64.exe`:
  1. Double-click it. If SmartScreen warns you, choose **More info**, then **Run anyway**.
  2. Click **Install**. No administrator rights are needed.
  3. Pollis opens when the installation finishes; later, open it from the Start menu.
- **Linux** (x64), `Pollis-linux-x64.tar.gz`:
  1. Run `tar -xzf Pollis-linux-x64.tar.gz`.
  2. Run `Pollis-linux-x64/pollis`.

> **The apps are not signed yet**
>
> That is why macOS needs the `xattr` step and Windows may warn you. Download Pollis only from the link above.

Install Julia

If Julia is not installed, the Welcome page says so under **Alerts and Announcements**: *Julia is not installed*.

1. Choose **Install Julia**, then **Download and Install**.
2. Pollis installs it with juliaup, the official Julia version manager, in a terminal. The terminal says when Juliaup and Julia have been installed.
3. The Welcome page then says *Julia … is ready*, and the Julia REPL starts.

If Julia is already installed somewhere Pollis cannot find it, choose **Set Julia Path** instead, and give the path of the Julia program in the setting `julia.executablePath`.

The first start of the Julia REPL

When Pollis opens, the Julia REPL starts in the panel at the bottom. The first time, it installs DuckDB and PollisDatasets for the built-in database, and says *This happens only once*. They go into an environment of Pollis's own, `~/.pollis/julia`, which comes after yours: your own versions of these packages are used first. At later starts, PollisDatasets is updated from GitHub when there is a newer version; offline, the installed copy is kept.

The packages a panel needs

The **Powered by:** line at the top of a panel names its Julia packages. Beside it, a tick means they are all installed; **Install** means some are missing, and installs them in the Julia REPL, in your active Julia environment. You do not have to: the code a panel sends to the Julia REPL or to a notebook starts with a line such as

```
Pollis.packages("GLM", "Plots", "PollisDatasets")
```

which installs whichever of them Julia cannot load, and does nothing when all are there. Replications, notebooks and Code Examples begin the same way. The first run of a panel can therefore take a few minutes; the next ones do not.

Updates

Pollis asks GitHub once a day for a newer release. When there is one, a badge appears on the **Manage** gear, and **Download Update** opens the file for your system in your browser: install it as above. At the first start of a new version, Pollis offers **What's New**; **Help > What's New in Pollis** opens the same notes at any time. To stop the checks, set `update.mode` to `none`.

A faster start

If you do not want the Julia REPL at every start, clear `julia.startREPLOnStartup`. It then starts the first time you send code to it.

If something goes wrong

macOS: "Pollis is damaged and can't be opened"	Run `xattr -cr /Applications/Pollis.app` in Terminal, then open Pollis again.
Windows: "Windows protected your PC"	Choose **More info**, then **Run anyway**.
***Julia is not installed*, though it is**	Choose **Set Julia Path** and give the path of the Julia program.
The first run of a panel seems stuck	Look at the Julia REPL: packages are being installed and compiled. Wait for the prompt to come back.
Install stays after installing	The tick appears once Julia has finished; it checks again every few seconds.

Not available yet

A Mac build for Intel processors, and signed apps.

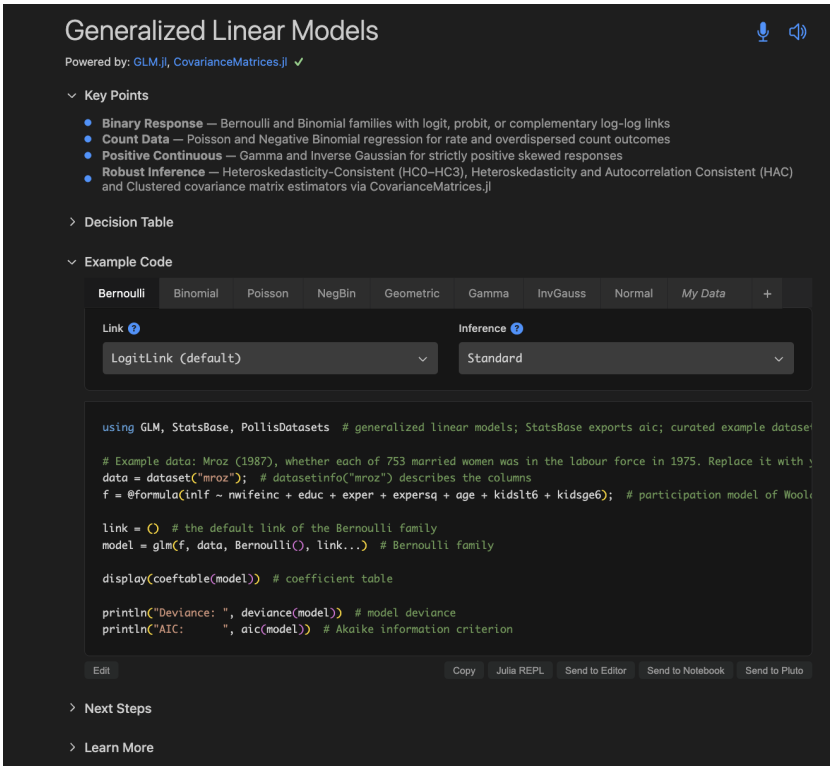
See also Inside a Panel, page 4. Data and Databases, page 6.

Inside a Panel

Explore · Model · Simulate · Optimise

What a panel is made of, and how to go from its example to code of your own.

Each item of the **Explore**, **Model**, **Simulate** and **Optimise** menus opens a *panel*: a page about one method, with an example you can run as it is, with simulated data or a dataset from PollisDatasets, and then make your own.



From top to bottom

- **The title**, and the **Powered by:** line: the Julia packages the panel uses, each a link to its documentation, with the tick or **Install** (see Getting Started, page 2).
- **Key Points**: what the method does and when to use it, in a few lines.
- **Decision Table**, in some panels: which variant to use for which data.
- **Illustration**, in some panels: a picture of the method or its variants.
- **Example Code**: the tabs of the panel's variants (above, **Bernoulli**, **Binomial**, …), its inputs, and the code. Changing an input changes the code at once.
- **Next Steps** and **Learn More**, at the bottom (next page).

Click a section's name to open or close it.

Run the example

The buttons below the code send it where you want it:

- **Julia REPL** runs it in the Julia REPL, starting it if needed.
- **Send to Notebook** opens it in a new Jupyter notebook, with the Julia kernel.
- **Send to Pluto** opens it in a new Pluto notebook, in Pollis's browser.
- **Send to Editor** opens it in a new, unsaved Julia file, and **Copy** copies it.

Code sent to the Julia REPL or to a notebook starts with the line that installs any missing package.

Make the example your own

1. Choose **Edit** below the code, and change it: your data, your variables, your options.
2. Choose **Save**, or **Cancel** to leave it as it was. A saved example is marked **Customised**, and the panel shows your version from then on.
3. To go back to the original, choose **Restore Default**, then **Confirm Restore** within four seconds. Your version goes to the Trash.

Add a tab of your own

1. Choose **+** at the end of the tabs, type a name and press **Enter**.
2. The new tab, marked **Added**, opens empty, for editing: write or paste your code, and **Save**.
3. To remove it, choose the **×** on the tab, then **Delete**; it goes to the Trash.

Your examples and tabs are files in `~/.pollis/examples`, one folder per panel. They are kept when Pollis is updated.

Next Steps

Visualise, Diagnose, Predict, Compare, Interpret: examples that carry on from the Example Code, using its results. Next Steps opens once you have run or sent the Example Code of the tab (the **?** beside it says so). Its examples have the same buttons, and can be edited and restored in the same way.

Learn More

- **Local Wikis**: a reference page on the method, in Pollis.
- **Notebook Tutorials**: notebooks that work through the method.
- **Multimedia Tutorials**: videos and talks. **Add / Edit Video** adds your own.
- **Explore References**: the papers and books behind the method, with **Open in Browser** and **Copy BibTeX to Clipboard**. **Add / Edit Reference** adds your own, from BibTeX, a link or a DOI.

> **The example data are an example**
>
> They show how the method works. To use it on your own data, change the lines that load the data, and check that the variables and options still suit your question.

If something goes wrong

Next Steps is dimmed	Run or send the Example Code of this tab first.
An error that a package is not found	Use **Julia REPL** or **Send to Notebook**, which install it, or **Install** on the **Powered by:** line.
The send buttons are greyed out	Finish editing (**Save** or **Cancel**); an added tab needs some code first.

> **Not available yet**
>
> The microphone and speaker buttons beside the title: voice comes in a later version.

See also Getting Started, page 2. Data and Databases, page 6. Customisation, page 8.

Data and Databases

Databases view · Explore > Access Files and Databases

The datasets that come with Pollis, your own databases, and the Databases view.

Pollis comes with a collection of real datasets, PollisDatasets, that its examples use, and connects to DuckDB, SQLite and PostgreSQL databases. All the work with data is done in Julia: the Databases view shows what is there and writes the code to reach it.

PollisDatasets

54 real datasets, such as `mroz`, `airpassengers`, `iris`, and `challenger`, each with its source and licence. In Julia:

- `datasets()` lists them;
- `datasetinfo("mroz")` describes one and its columns;
- `data = dataset("mroz")` loads one as a Tables.jl table.

They are installed the first time the Julia REPL starts (Getting Started, page 2). When new datasets are added, the Welcome page says so.

The built-in database

The same datasets are also a DuckDB database, *PollisDatasets*, one table per dataset, named after it, with a description of the table and of each column. It is read-only, and is rebuilt only when PollisDatasets changes. The Welcome page says when it is ready.

1. Open the **Databases** view, in the secondary side bar on the right (or choose **Open Databases View** on the Welcome page).
2. Click **PollisDatasets**, marked **built-in**. It connects in the Julia REPL as `pollis`.
3. Expand it to see its tables and their columns.

Then query it in the Julia REPL with SQL:

```
DBInterface.execute(pollis, "SELECT * FROM mroz WHERE educ > 12")
```

The Databases view

- Click a connection to connect it in the Julia REPL; **Disconnect** closes it.
- On a table, **Open in Table Viewer** shows its rows; right-click it for **Preview Rows in Julia REPL**.
- **Show Tables** and **Hide Tables**, beside a connection, expand or collapse it; **Refresh Schemas**, at the top of the view, shows tables you have just created.
- Right-click a connection for **Copy Name**, **Copy Connection Code**, **Edit Connection** and **Delete Connection**, which removes only the saved connection, never the database.

Connect to your own database

1. In the Databases view, choose **New**, then **DuckDB**, **PostgreSQL** or **SQLite**.
2. Give a **Connection Name** and the **Julia Variable** that will hold the connection, and choose whether to keep it for all your work (**Global**) or for this folder only (**Current Workspace**).
3. For DuckDB or SQLite, choose the database file (**Choose File**). A DuckDB connection with no file is an empty database in memory: it can read CSV and Parquet files in place, and is lost when you disconnect. **Read Only** protects a file from changes.
4. For PostgreSQL, give the **Host**, **Port**, **Database** and **Username**.
5. Check the **Generated Code**, then **Save Connection**, or **Connect** to save it and connect.

A connection opened by your own code can be shown in the view as well:

```
con = PollisDB.show(DBInterface.connect(DuckDB.DB, "sales.duckdb"); name = "Sales")
```

> **PostgreSQL passwords**
>
> With **Ask for Password in the REPL** (the default), you type the password in the Julia REPL when you connect: it is never written in the code, in the REPL history or in a file. Without it, the password comes from `PGPASSWORD` or `~/.pgpass`.

Panels for data

- **Explore > Access Files and Databases**: **SQL Databases** (query, import a CSV, Parquet or JSON file, save a table, export a query), **CSV Files** and **Excel Workbooks**.
- **Explore > Access Data Libraries**: Hugging Face and Kaggle datasets, Yahoo Finance, Alpha Vantage, US SEC EDGAR, FRED and the European Central Bank. A provider that needs an API key shows **Set API key** on the **Powered by:** line; the key is kept in your system's secure storage.

If something goes wrong

The view shows no tables	Choose **Refresh Schemas**. A new database stays empty until you create a table in the Julia REPL.
PollisDatasets does not connect	The Julia REPL has not started yet, or is still installing: wait for the Welcome page to say the database is ready.
Cannot write to `pollis`	The built-in database is read-only: save your tables in a database of your own.
PostgreSQL does not connect	Check the host, port and **SSL Mode**; the **Connection Timeout** is 10 seconds.

> **Not available yet**
>
> International Monetary Fund Datasets, World Bank Indicators and the Kenneth French Data Library are in **Access Data Libraries**, greyed out.

See also Getting Started, page 2. Collaboration, page 24, for sharing datasets and databases.

Customisation

Pollis Views · Settings

Showing only the menus you need, and making Pollis yours.

Pollis has hundreds of panels. A *View* is a named set of the menu items Pollis shows: a course can keep only its own topics, a researcher only their field. The panels you hide are still there; another View shows them again.

Choose a View

The name of the View in use is in the title bar, before the layout buttons: **View: Max**. Click it for the Views menu, with a tick on the one in use. Two Views are built in:

- **Max** shows every menu item. It is the default.
- **Min** shows a short selection: the common plots and Descriptive Statistics in **Explore**, and a few models, simulations and optimisation panels.

Switching applies at once, to every menu. The built-in Views cannot be changed or deleted.

Make a View of your own

1. In the Views menu, choose **Create View...**, type its name, and choose the View it starts from: **Max** starts with every menu item. Or choose **Save View As...** to copy the View in use.
2. The new View becomes the one in use, and the **Views** panel opens in the secondary side bar, on the right, after **Databases**: the menus, with every item below them.
3. Untick an item to hide it, tick it to show it. Unticking a submenu hides everything in it; a submenu shows how many of its items are shown, such as **4/12**. Changes apply at once.

Explore, **Model**, **Simulate**, **Optimise**, **Extend** and **Compose** can be trimmed to the items you tick; in the other menus, such as **File** or **Help**, you can hide single items.

Edit, delete and share Views

- **Edit View...** makes one of your Views the one in use, and shows its items in the **Views** panel.
- **Delete View...** moves its file to the Trash, so it can be restored. Deleting the View in use switches to **Max**.
- **Open Views Folder** opens `~/.pollis/views`: each of your Views is a JSON file there. Copy a file to another computer, or give it to your students, to use the same View.

The View in use is kept in the setting `pollis.view`.

Settings

Choose the **Manage** gear at the bottom left, then **Settings**, and type in the search box a word or a setting's id, such as `julia.startREPLOnStartup`. **Themes > Color Theme**, in the same gear, changes the colours, light or dark.

Extensions

The Extensions pane, in the activity bar on the left, installs extensions from Open VSX, the open registry of extensions, as well as Pollis toolboxes (next topic). Some extensions published only by Microsoft are not on Open VSX, so they cannot be installed in Pollis.

What Pollis keeps for you

Your own things are files in `~/.pollis`, kept when Pollis is updated:

`views`	your Views
`examples`	your Example Code and added tabs (Inside a Panel, page 4)
`workflows`	your copies of the replications (page 14)
`methods`, `packages`	your new methods and their packages (page 16)
`practice`	the exercises you write (page 22)

> **A hidden item is only hidden**
>
> A View changes the menus, nothing else: the Command Palette and links between panels still reach every panel. A View is not a way to stop students from opening a panel.

If something goes wrong

A menu item has gone	The View in use hides it: switch to **Max**, or tick it in your View.
The items cannot be ticked	The View in use is built in: choose **Save View As...** to make an editable copy.
An extension is not found	It is not on Open VSX. Look for an open alternative there.

See also Toolboxes, page 10. Inside a Panel, page 4.

Toolboxes

Extensions pane > Toolboxes · Compose > New Panels and Toolboxes

Adding a whole field to Pollis with a toolbox extension.

A toolbox is an extension that adds a set of panels to the menus, with their wikis and notebook tutorials. It holds no program code: its panels are made like Pollis's own. Three are published:

- **Financial Econometrics**: ARIMAX, VAR and VECM, GARCH-type and regime switching models, forecasting, Bayesian asset-pricing factor models.
- **Economics**: business cycle filters, stochastic volatility, DSGE and heterogeneous-agent models, continuous-time macro-finance, game theory.
- **Mathematical Core**: symbolic maths, linear algebra, nonlinear systems, numerical differentiation and integration, ordinary and partial differential equations.

Install a toolbox

1. Open the Extensions pane, in the activity bar on the left. The **Toolboxes** section, at the top, lists the toolboxes you have not installed. Click a name for its details.
2. Choose **Install**. When Pollis asks, choose **Reload Now** or **Restart Extensions**.
3. The **Toolbox Panels** page opens: installing a toolbox does not change your menus until you choose its panels there (next page).

An installed toolbox moves to the **Installed** section. When a newer version is published, it comes back to **Toolboxes** with **Update**.

Choose its panels, and where they go

The **Toolbox Panels** page shows the toolbox on the left and **Your Menus (Preview)** on the right.

1. Tick the panels you want (**Select All**, **Select None**), or choose **Display Author's Suggestion** to see where the author would put them.
2. Click a blue line in the preview, **Put here**, to put the ticked items there. A ticked toolbox menu moves with its panels inside; untick it to move panels out of it.
3. Choose **Publish**. The page then says **In your menus as shown.**

A toolbox can go in **Explore**, **Model**, **Simulate**, **Optimise** or their submenus, or in **Extend**, a menu that appears only when a toolbox puts something in it.

To change them later, choose **Edit** on the toolbox in the **Installed** section, or **Toolbox Panels...** in the Command Palette (**Compose: Toolbox Panels...**). To remove a toolbox, choose **Uninstall** on it in the same section.

Make a toolbox

A teacher can package the panels of a course as one toolbox and give it to the students. This is done by your coding agent (Orchestration, page 26), with a skill that writes the toolbox for you.

1. Choose **Compose > New Panels and Toolboxes > Package a Toolbox...** The agent starts in a terminal.
2. Tell it which panels you want, by their menu titles, the menus they go in and their order, and the toolbox's name and description. It asks for what is missing.
3. It writes a short spec, builds the toolbox and its README, and says where the `.vsix` file is.
4. To install it, run **Extensions: Install from VSIX...** in the Command Palette and choose the file. Your students do the same with the file you give them. Then choose its panels, as on the previous page.

To change it, edit the spec, raise its version, build again and install the new file over the old one.

New panels

Compose > New Panels and Toolboxes > Create a Panel... has your coding agent write a new panel with you. **Create Panels from a Request...** writes several at once from a request file, which lists the toolbox and, for each panel, its topic, methods and menu. Without a request file open, it opens an example to fill in. New panels can then be packaged as a toolbox.

> **Toolboxes need the Internet to install**
>
> The list of toolboxes and their files come from GitHub. Once installed, a toolbox works offline.

If something goes wrong

"Could not load the list of toolboxes"	Check the Internet connection, then choose **Retry**.
Installed, but nothing in the menus	Choose its panels and **Publish**: **Edit** on the toolbox, in **Installed**.
The Toolbox Panels page is empty	Reload the window or restart the extensions, as Pollis asked, first.
A panel cannot go in a toolbox	Some Pollis panels have code of their own and are marked not portable; the agent says which.

See also Customisation, page 8. Orchestration, page 26.

Replication, Methods and Workflows

Replay a classic. Keep what it teaches.

Besides its replication record, a replicated article can leave two things behind: a new method, and a workflow.

1 Replication

It starts from an article, its input data, and either the results it publishes or its authors' code, in any language. If a method the article uses is missing from Julia's packages, it is written first. Then each result is printed beside the published one. Replication, page 14.

2 Methods

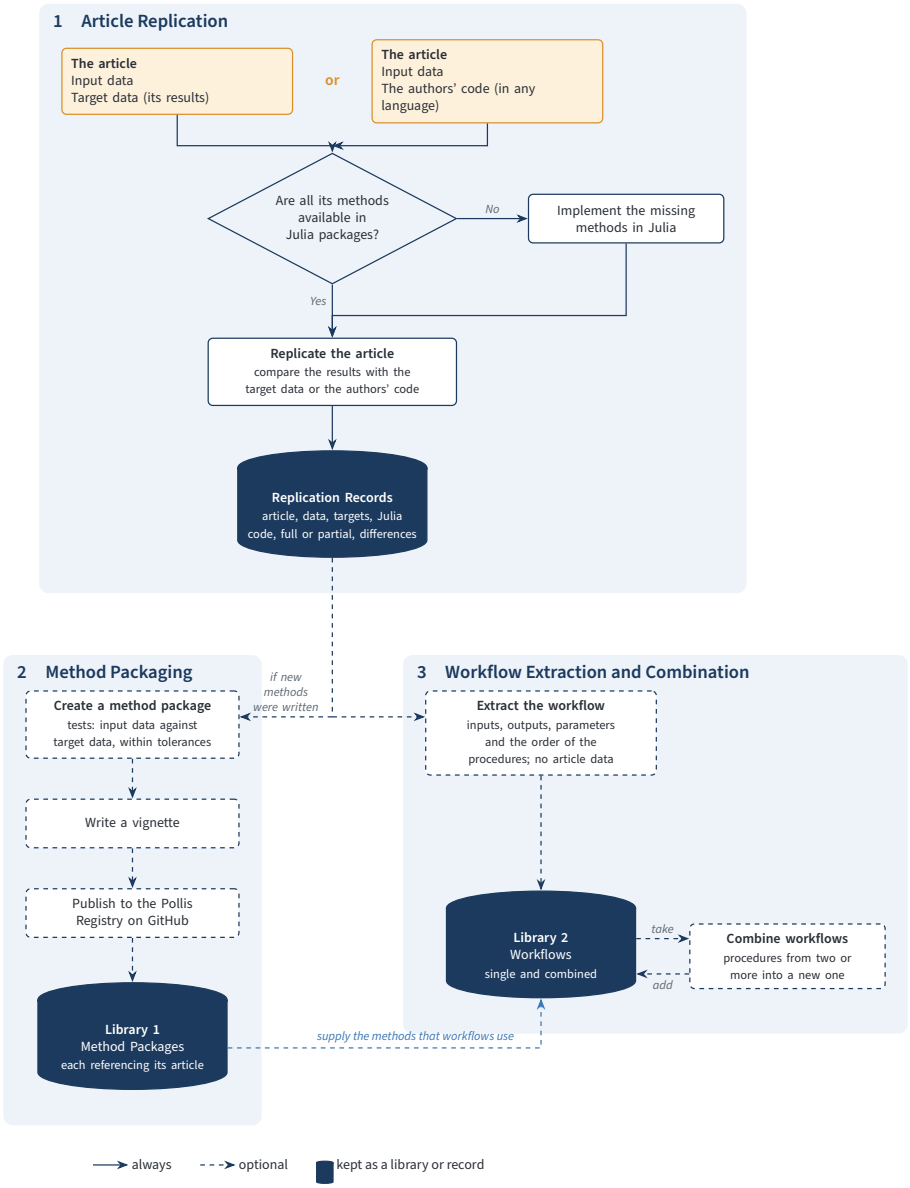
A method an article needed and no Julia package had becomes a documented Julia package. Its tests run the article's data and check the published results, within tolerances. It comes with a vignette, and can be published in the Pollis registry. Methods, page 16.

3 Workflows

A flat set of procedures, each with its inputs, its outputs and what it runs. Extracted from a replication, it runs on your own data; combined with others, its independent procedures run side by side.

Not available yet

Replication records (the article, its data, the targets, the Julia code, whether the replication is full or partial, and any differences); **Compose > Replication > Replicate a Classical Article...**; and **Browse Workflows...**, **Extract a Workflow...** and **Combine Workflows...** in **Compose > Workflows**.



Replication

Compose > Replication

Replaying a classic paper from its own data, step by step.

A *replication* reproduces a published analysis on its own public data: a Jupyter notebook that follows the article section by section and prints every published value beside Julia's. It shows a method at work on a real question, and that Julia gets the published answer.

The replications

Replication	Article
Panel data: is public capital productive?	Munnell (1990)
GARCH: volatility after non-trading days	Bollerslev and Ghysels (1996)
Survival: the Mayo model for primary biliary cirrhosis	Dickson et al. (1989)
Logistic regression: the Challenger O-rings	Dalal, Fowlkes and Hoadley (1989)
Threshold autoregression: the lynx and the sunspot cycles	Tong and Lim (1980)
Dynamic time warping: the UCR archive baselines	Dau et al. (2019)
Propensity score matching: the NSW experiment and its replications	Dehejia and Wahba (1999)
Random forests: Breiman (2001)	Breiman (2001)
LSTNet: long- and short-term patterns in multivariate time series	Lai, Chang, Yang and Liu (2018)

Open a replication

1. Choose **Compose > Replication > Browse Replications...**
2. The page lists the replications with their article (its DOI, or **Web Page** when it has none, and **Copy BibTeX**), the Julia packages and datasets they use, their size, any data they download, and the date they were last updated. Click **more** under a description to read it all.
3. Click a replication's name. The first time, Pollis makes your own copy in `~/.pollis/workflows`, and opens its notebook.

Your copy is yours to change: later openings show it as you left it.

Follow it

1. Choose the Julia kernel if the notebook asks for one.
2. Run the cells in order, from the top. The first code cell, `Pollis.packages(...)`, installs any package that is missing, so the first run takes longer.
3. Read each section beside the article: the text says which table, figure or page it replicates.

The data come from PollisDatasets or from the source the article used. The page says when a replication downloads data, and how much (**+ *size* download**); **download if asked** marks data that it downloads only when you agree.

Checking the results

Most comparisons are small tables: Julia's value in full, the same value rounded as the article prints it, the published value, and the number of decimals they share, or `DIFFERENT`. Each table's title says where the published value is printed, usually a page or table. Random forests averages 100 random runs, as Breiman did, so it asks instead whether Julia's average and Breiman's differ by less than two standard errors. All the notebooks but the panel-data one end with a section, *What differs, and why*.

New and updated replications

New replications, and new versions of the ones you have, can arrive between releases of Pollis: the page marks them **New** or **Updated** until you open them. When your copy is older than the new version, the page says *Your copy is older than this version* and offers **Open Updated Version**: the new notebook is saved beside yours, with its date in the name, and your copy is kept.

From a replication to a report

A replication's notebook can become a LaTeX report or slides: Reports, page 20. The GARCH replication has its own report template.

If something goes wrong

A cell fails: a package is not found	Run the first code cell, `Pollis.packages(...)`, then the others in order.
A download fails	Check the Internet connection; the source may be down for a while. Try again later.
You changed your copy and want the original	Rename or delete its folder in `~/.pollis/workflows`; opening the replication again makes a fresh copy.

Not available yet

Replicate a Classical Article... is in the menu but not built yet: it says so when chosen. So are **Browse Workflows...**, **Extract a Workflow...**, **Combine Workflows...**, **Create Operational Workflow...** and **Automations...**, in **Compose > Workflows**.

See also Reports, page 20. Methods, page 16. Data and Databases, page 6.

Methods

Compose > Methods

Turning a new method into a tested Julia package, with its vignette.

A *new method* is one no Julia package has yet, taken from an article: its data, the results the article publishes for them, and the code that reproduces them. Pollis makes it a Julia package whose tests are the published values, with a *vignette*, its documentation, and can publish it in the Pollis registry. Without data and expected results there is no new method.

Start a method

Choose **Compose > Under Construction...**. The method's page has five numbered steps, below; its title is at the top: click it to change it. Nothing is saved until you type or add something. Each method is a folder in `~/.pollis/methods`.

1. Article and Data

- **Article**: the **Article Link**. Pollis opens it in the integrated browser and keeps no copy; **Save a Copy...** saves one outside Pollis, by default in the `data` folder of your Pollis Workspace (Reports, page 20).
- **Input Tables**, the data the method is computed from: **Empty Table** to type one in, **Add from a Link**, **Add from PollisDatasets**, or **Add from Article**: give the numbers of the article's tables, such as `3.2, 5.8`, and click **Ask Agent**.
- **Target Tables**, the published results the method must reproduce: **Empty Table**, **Add from CSV Files**, or **Add from Article**. Every printed value becomes a test of the package.

A table typed in by the agent keeps every value as printed, and the agent lists the values it could not read for certain: check them against the article. The tests are only as right as the targets.

2. Write a Method

Write **Instructions** for the agent, saved in `INSTRUCTIONS.md`: what the method does, where it is in the article, what to watch for. Then click **Write Method**: your coding agent writes the code in `src`, with you, and runs the tests. **Edit method.toml** shows the method's own file: its tables, its call and its article. **Open Code** opens the code.

3. Write a Vignette

Once the method's tests pass, write notes on the vignette (`VIGNETTE.md`) and click **Write Vignette**: the agent writes it in LaTeX, with its PDF. **Edit Vignette** and **Open PDF** open them.

4. Prepare for Registration

Give the **Package Name**: a capital letter, then letters and digits, such as `ThresholdAR`; the Pollis registry asks for at least 5 characters. It also asks for a usage example and a docstring for every exported name: **Prepare for Registration** has the agent check the method and fix what it can.

5. Package and Publish

Each button runs in a terminal. **Package and Test** makes the package in `~/.pollis/packages`, runs its tests and checks the registry rules. **Package, Test and Push to GitHub** then pushes it to its own public repository, if its tests all pass. **Package, Test, Push and Register** then registers it, if it meets the rules. Pushing needs a **GitHub Owner**, a user or organization, and the GitHub CLI, `gh`, signed in; Pollis asks first: *Push to GitHub as … ?*

Browse methods and read vignettes

Compose > Methods > Browse Methods lists yours, **In Progress** and **Published by You**, then the **Library** of nine methods by Pollis, from arc-fs to threshold autoregression: click one to open your own copy. **Delete** and **Delete Empty Methods** move folders to the Trash. **Help > Documentation > Vignettes** lists the vignettes of installed packages (they open offline) and, with the Internet, those **In the Registry**.

> **The agent steps cost tokens**
>
> Add from Article, Write Method, Write Vignette and Prepare for Registration run your coding agent, on your account.

If something goes wrong

… has no target tables, so nothing to test	Add the published results in **Target Tables**.
Copies are kept outside Pollis	Save the copy outside `~/.pollis`, e.g. in a Pollis Workspace.
The push fails	Sign in with `gh auth login` in a terminal, and check the GitHub Owner.

See also Replication, page 14. Orchestration, page 26.

Pollis Packages

github.com/Trumpingtons

The Julia packages written for Pollis, which you can also use without it.

Most panels use Julia packages from Julia's General registry. Fourteen packages are Pollis's own: nine methods, and five packages behind panels and features. All are public on GitHub, under Trumpingtons, with the MIT licence, and work in any Julia session, with or without Pollis. None is in the General registry.

The method packages

The nine methods of the **Library** in **Compose > Methods > Browse Methods** (Methods, page 16). Each is tested against the published results of the article it comes from, has a vignette, and is in the Pollis registry.

Package	Topic	Tested against
ArcFS	Boosting	Breiman's arc-fs, from his random forests article
BreslowCox	Survival analysis	The Mayo model of primary biliary cirrhosis
ErrorComponents	Panel econometrics	Baltagi's tables for the Grunfeld, gasoline and public capital data
ForestOOB	Random forests	Breiman's out-of-bag errors and variable importance
GLMExtras	GLM inference	Dalal, Fowlkes and Hoadley's Challenger O-ring analysis
LSTNet	Time series forecasting	Lai, Chang, Yang and Liu's LSTNet-Skip and their PyTorch code
PeriodicGARCH	Volatility	Bollerslev and Ghysels's periodic GARCH
PropensityMatching	K-nearest neighbours	Dehejia and Wahba, and its replications
ThresholdAR	Nonlinear dynamics	Tong and Lim's threshold autoregression

Install a method package

Add the Pollis registry once, then add packages by name, in the Julia REPL's package mode (type]):

```
pkg> registry add https://github.com/Trumpingtons/PollisRegistry
pkg> add ThresholdAR
```

Help > Documentation > Vignettes opens the vignettes of the packages you installed.

Packages behind panels and features

Package	What it does	Where Pollis uses it
PollisDatasets	54 small, well-known datasets for examples and teaching, as Tables.jl column tables	The built-in database (Data and Databases, page 6), **Add from PollisDatasets** in Methods, and the examples of panels and replications
PollisTeX	Typesets LaTeX from Julia with Tectonic, downloaded on first use	**Compose > Reports** (Reports, page 20)
EDGAR	Fetches SEC EDGAR filings and financial data	**Explore > Access Data Libraries > US SEC EDGAR**
HARE	Heteroskedasticity and autocorrelation estimators	**Model > Linear Models**: **Linear Models with Heteroskedasticity**, **with Autocorrelation** and **with Heteroskedasticity + AR(1)**
RangeVol	Range-based volatility estimators	The Range-Based Volatility Estimation panel, not in the menus at present

Install one

Pollis installs these packages by itself when a panel, a replication or a report needs them. To install one yourself, give its GitHub address, in the Julia REPL:

```
import Pkg; Pkg.add(url = "https://github.com/Trumpingtons/HARE.jl")
```

The datasets' own terms

The MIT licence covers PollisDatasets' code. Each dataset keeps the terms of its provider: `datasetinfo("penguins")` shows its description, the source to cite, its licence and its columns.

If something goes wrong

`add ThresholdAR`**: the package is not found** Add the Pollis registry first, as above.

`Pkg.add("HARE")`**: the package is not found** These five are in no registry: install with `Pkg.add(url = ...)`, as above.

See also Methods, page 16. Data and Databases, page 6. Reports, page 20.

Reports

Compose > Reports

From a notebook to a LaTeX report or slides.

Any Jupyter notebook, a replication's or your own, can become a LaTeX report (an article) or beamer slides, with its outputs and figures and a PDF beside it. Pollis does this in its Julia REPL, without an agent; a coding agent can then polish the draft into a finished document.

Before you start: a Pollis Workspace

Reports are written in the `reports` folder of a *Pollis Workspace*: an ordinary folder for your own work, outside Pollis. To make one, choose **File > New Pollis Workspace...**, then choose or create a folder, for example in your Documents, and click **Make Pollis Workspace**. Pollis puts a `pollis.toml` file and a `data` folder in it, and opens it. Next time, open that folder with **File > Open Folder...**.

Make a report or slides

1. Open the notebook, for example your copy of a replication (page 14).
2. Choose **Compose > Reports > Export Notebook as LaTeX...** for a report, or **Make Slides from a Notebook...** for slides. The notebook's own toolbar has the same two in its **Export** button: **LaTeX Report...** and **Beamer Slides...**. With no notebook open, Pollis asks you to choose one, starting in `~/.pollis/workflows`.
3. The Julia REPL installs PollisTeX if it is missing, runs the notebook's code cells in order and compiles the document.
4. The `.tex` opens in the editor, and the PDF beside it, in the integrated browser.

The files are in `reports/`*`NAME`* of the workspace, *NAME* being the notebook's: `report.tex` or `slides.tex` and its PDF, each cell's printed output in `outputs/cellN.txt`, and its figure in `figures/cellN.pdf`.

What the draft holds

- A *report* is a draft of the notebook's text cells, with the printed outputs and figures in their place and without the code.
- *Slides* have a frame for the text under each heading, and one for each output and each figure.
- A notebook with a *template*, a `latex` folder beside it, gets that document instead, with its own tables, figures and references. The GARCH replication ships one for its report.

Edit and compile

Edit the `.tex` as you like, then click **Compile LaTeX**, the play button at the top right of its editor: Pollis saves it, compiles it with PollisTeX in the Julia REPL and reloads the PDF. Exporting the notebook again refreshes the outputs and figures, and keeps your `.tex`: Pollis writes it only when the folder has none. To start the draft again, delete the `.tex` first.

Polish with Agent

1. Open the report's or the slides' `.tex`.
2. Choose **Compose > Reports > Polish with Agent...** The coding agent starts in a terminal in the report's folder.
3. It turns the draft into a finished document: sections, tables, captions and citations, from the numbers already computed. It does not run the notebook again, and it compiles the document as it goes: the PDF beside you reloads each time.

Polish runs on the Light tier of Model Routing (Orchestration, page 26).

> **Polish costs tokens**
>
> The menu item with the sparkle runs your coding agent, on your account. Read the result: an agent can still misplace a value, so check the numbers against the draft.

If something goes wrong

Reports are written in the reports folder of a Pollis Workspace	Click **New Pollis Workspace...**, or open your workspace folder.
A Pollis Workspace keeps your work outside Pollis	Choose a folder outside `~/.pollis`, for example in your Documents.
No PDF appears	A cell or the compile failed: read the error in the Julia REPL, fix it, and export or compile again.
Polish says to open the .tex	Open `report.tex` or `slides.tex` first, then choose **Polish with Agent...** again.

See also Replication, page 14. Orchestration, page 26.

Practice and Learning Activities

Help > Practice · Compose > Learning Activities

Practising a topic, and writing exercises for others.

Pollis's exercises are multiple-choice questions whose data and answers Julia computes, so each time you practise, the numbers are new. Pollis comes with three topics, *Descriptive Statistics*, *Hypothesis Tests* and *Linear Regression*, of five exercises each; the exercises you or your teacher write are listed beside them.

Practise a topic

1. Choose **Help > Practice > Practise a Topic...** The page lists the topics, with their exercises and their **Source**: **Pollis**, **Yours**, or **Pollis and yours**.
2. Click a topic, or **Practise All** under it, to practise its exercises in order, or click an exercise to start from it.
3. Read the question, pick a choice and click **Check**: Pollis says *Correct.* or *Not quite: the answer is . . .*, with an **Explanation**.
4. **Next** and **Previous** move between exercises; you can skip one and come back to it. **New Data** asks the same question on new numbers. **Learn More**, when the exercise has one, opens the panel that explains the method.
5. **See Score** gives your score, such as *You answered 4 of 5 correctly, and skipped 1.* **Practise Again** starts again, on new data.

The first time, Julia may install a package an exercise needs, so making the exercises takes longer.

Not available yet

Take a Practice Test..., **Try a Timed Challenge...**, **Ask a Tutor...** and **My Progress...** in **Help > Practice** are in the menu but not built yet: they say so when chosen.

Write an exercise

1. Choose **Compose > Learning Activities > Write an Exercise...**
2. Pick its topic, or **New Topic...** and type the topic's title, such as *Probability Distributions*. Then type the exercise's title.
3. Pollis writes the exercise in `~/.pollis/practice`, one folder per topic, from an example that works, and opens it. A preview opens beside it.
4. Change the example and save: each save shows a new version in the preview, or the error Julia raised.

An exercise is a short TOML file:

`title`	the exercise's title
`panel`	optional: the command of the panel that **Learn More** opens
`packages`	the Julia packages it needs, installed when missing
`setup`	Julia code that makes the data, with `rng`, a random number generator
`question`	the question
`choices`	the choices; Pollis shuffles them
`answer`	the correct choice, counted from 1
`explanation`	what the learner reads after **Check**

In the text, `{{...}}` is Julia: `{{fmt(mean(x), 2)}}` writes the mean of `x` with 2 decimals, `{{list(x)}}` its values separated by commas. Make the wrong choices typical mistakes, such as dividing by $n - 1$ for a mean. When two choices come out the same, Pollis draws the data again, so a choice is never ambiguous.

> **Check every version**
>
> Julia computes the answers, but you wrote the question: save a few times, and check that each version's correct choice is right and the others are wrong.

If something goes wrong

Julia could not make the exercises	Check that Julia is installed, or set its path in `julia.executablePath`.
Left out, because Julia raised an error	Open the exercise and fix it: the preview shows the error.

> **Not available yet**
>
> **Write Exercises with My Agent...**, **Create a Test...**, **Browse Tests...** and **Export to a Learning Platform...** are in the menu but not built yet.

See also Inside a Panel, page 4.

Collaboration

Compose > Collaboration

Working with others at their own computers: a chat, and sharing code, datasets, files and databases.

A session connects up to five people, each at their own copy of Pollis: two colleagues, a student and a tutor, or a small team. Everyone keeps their own editors and notebooks; in the chat you talk, and you share what you choose, when you choose. Nobody can change anything on anyone else's computer.

What you need

- Pollis on each computer, and an Internet connection.
- A network that lets sessions through. To check, choose **Compose > Collaboration > Test Connection...** before the session, and send its results (**Copy Results**) if you need help.

Start a session

1. Choose **Compose > Collaboration > Start Side by Side Session...**
2. Type your name, as the others will see it. The chat opens beside your editor, and the invitation is copied to the clipboard.
3. Paste the invitation into an email or a message to the others. To copy it again, use **Copy Invitation** at the top of the chat, or **Compose > Collaboration > Invite People...**

You are the *host*: you admit the others, up to five in all, and when you leave the session ends for everyone.

Join a session

1. Choose **Compose > Collaboration > Join Session...**
2. Paste the invitation you received, or type its code: four groups of five letters and digits, such as `7KQ2M-....`
3. Type your name. The chat opens and says when you have joined.

The chat

Write a message and press **Enter**; **Shift+Enter** starts a new line. Paste or drag an image into the message: larger images are reduced to fit the limit of 2 MB. The **+** button in the message box adds:

- **Image**: a PNG, JPEG, GIF or WebP image, shown in the chat.
- **File**: any file up to 5 MB. The others get a card with **Save As...**; Pollis never opens or runs it. Dragging a file that is not an image into the chat shares it the same way.
- **Dataset**: a CSV file up to 5 MB (see the next page).
- **Database**: a DuckDB database up to 5 MB (see the next page).

Show Participants lists who is in the session and whether each connection is direct or through a relay.

> **The invitation is the key**
>
> Anyone with the invitation can join while the session lasts, and receives whatever is shared from then on. Send it only to the people you mean, and do not share personal or confidential data in a session whose invitation may have gone further.

Share your code

While you are in a session, a bar at the bottom of each editor and notebook shows **Share My Editor**. It sends a snapshot of what you see, unsaved changes included; a notebook is sent without its outputs. For a Pluto notebook in Pollis's browser, use the **Share My Editor** button in the browser's toolbar. Sharing again replaces the earlier snapshot. Code files and notebooks up to 2 MB can be shared.

The others see your snapshot read-only, with two buttons below it:

- **Compare with Mine**: your snapshot and their own editor side by side, line by line.
- **Copy to My Editor**: replaces the content of their own editor, or the cells of their own notebook, with yours. **Undo** brings theirs back, except in Pluto.

Share data

- **A dataset** (CSV): **+ > Dataset** in the chat, **Share Dataset** on a CSV file in the Explorer, or **Share Dataset** in the toolbar of a Julia table viewer. Pollis asks before sending. The others get a card with the first five rows and **Preview**, **Save to My Workspace** and **Load into My Julia Session**, which reads it into a Julia table named after the file.
- **A database** (DuckDB): **+ > Database**. Pollis asks before sending the whole database. The others get a card with **Save As...** and **Attach in My Julia Session**, which opens it *read-only*, as a variable named after the file, and shows its tables in the Databases view.

> **Before sharing a database**
>
> Changes made in a Julia session that still has the database open may not be in the file yet: close that connection, or run `CHECKPOINT`, before sharing. And attach only databases from people you trust: a database can hold views that read files on the computer where they are queried.

Leave or end the session

Leave Session, at the top of the chat or in **Compose > Collaboration**, leaves the session. When the host leaves, the session ends for everyone. If the host's connection drops, the others wait a minute for it to come back. After the session the chat stays open, to be read, until you close it.

How you are connected

Free public servers introduce the participants to each other; after that each pair connects directly. The servers never see the session code or anything you share. By default they are nostr relays; for networks that block those, set `pollis.collaboration.signallin` to `mqtt` before starting the session (the invitation tells the others). A relay for networks that block direct connections is not set up yet.

If something goes wrong

"Could not reach the session"	Check the code, and that the host is still in the session. Then run **Test Connection...** on both computers.
Test Connection: no introduction server reachable	The network blocks Pollis (often a firewall or proxy): try another network, such as a phone's hotspot.
Test Connection: nostr blocked, MQTT reachable	The host sets `pollis.collaboration.signalling` to `mqtt` and starts a new session.
Two people never connect	Both networks may block direct connections, which need the relay that is not set up yet: one of them tries another network.
"The session is full"	A session has at most five participants, the host included.
A file, dataset or database is refused	It is over 5 MB, or not what it says it is: send it the usual way.

> **Not available yet**
>
> **Start Same Page Session...** (editing together), **Invite an Agent...**, **Session Settings...**, **Classroom** and **Simulation Games** are in the menu but not built yet: they say so when chosen.

See also Data and Databases, page 6. Inside a Panel, page 4.

Orchestration

Compose > Orchestration · Compose > Coding Agents

Choosing and starting coding agents, and routing tasks to models.

Some tasks of the **Compose** menu are done by a *coding agent*, a program that writes code and documents in a terminal: creating panels, packaging toolboxes, writing methods and vignettes, polishing reports. Their menu items carry a sparkle, because they run on your agent's account and cost tokens. Pollis works with any agent: you choose which, and which model each task runs on.

Choose your coding agent

1. Choose **Compose > Coding Agents**. The page lists Claude Code, Codex, GitHub Copilot CLI and OpenCode, each with its command and whether it is **On This Computer**: **Agent Installed**, or **Install Agent**, which opens its install page.
2. Click **Use** beside yours; it then shows **In Use**.
3. Click **Install Skills**, so that the agent knows the Pollis skills in any folder, even outside Pollis. Later, **Update** installs their latest version.

Each agent also has a *local* command, under **Local and Cloud Models**, that runs it on a model on this computer through Ollama or LM Studio: free and private. For any **Other Agent**, type its command line with `{prompt}` where the first message goes; without it, Pollis copies the message to the clipboard, to paste into the agent. The choice is kept in `pollis.agents.command`.

What happens when a task starts

Pollis opens a terminal, starts the agent there and gives it a first message that points it to the skill for the task. From then on you talk to the agent in that terminal.

Agent Skills

A *skill* is the know-how an agent reads to do a task. **Compose > Agent Skills** lists them, downloaded in their latest version when the page opens: create-panel, package-toolbox, prepare-method, references, write-method and write-vignette, each with its **Tasks** and **Example Requests**. **View Skill** opens it; **Run with My Agent** starts your agent on it.

Model Routing

Not every task needs the strongest model. **Compose > Orchestration > Model Routing** has three *tiers*:

- **Light**: small tasks, on a fast, cheap or local model.
- **Standard**: everyday tasks.
- **Strong**: hard tasks, where a mistake costs most.

Give each tier the command line of an agent, with `{prompt}`, such as `claude -model haiku {prompt}` for Light. An empty tier runs your main agent, and so does a tier whose program is not installed. Each task runs on a tier by default:

Task	Menu	Tier
Polish with Agent	Reports	Light
Write a Vignette, Prepare for Registration	Methods	Standard
Package a Toolbox	New Panels and Toolboxes	Standard
Tables from the Article, Write a Method	Methods	Strong
Create a Panel, Create Panels from a Request	New Panels and Toolboxes	Strong

In **Tasks**, choose another tier for a task, or **Main Agent**. The settings are `pollis.agents.tiers` and `pollis.agents.taskTiers`.

Local Models

Compose > Local Models has **Hugging Face Models** and **Kaggle Models**, to find models, and **Local Model Runners**. A local model runs only through a *runner*, a program that loads it and serves it on this computer: Ollama, LM Studio, llama.cpp or Apple MLX. The page shows which are installed, and how each agent reaches each one.

> **An agent works on your files**
>
> A coding agent reads and writes files, and runs commands, in its terminal. Read what it proposes, and check its results, as you would a colleague's.

If something goes wrong

Choose the coding agent Pollis starts	Choose one in **Compose > Coding Agents**, then the task again.
The coding agent … was not found	Install it (**Open Install Page**), or choose another one.
… tier: … isn't installed	The task ran on the main agent: install the tier's program, or change its command.

Not available yet

Writer and Checker..., **Two Independent Transcriptions...** and **Race...** are in **Orchestration** but not built yet.

See also Toolboxes, page 10. Methods, page 16. Reports, page 20.